

Lista de Exercícios de Treino — Prova 1 (UFSM)

Baseada nas Provas Reais (2024 e 2026 - Prof. Mateus Beck Rutzig) e nos Slides Oficiais de Monociclo e Multiciclo.

BLOCO 1: Projeto de Novas Instruções no Monociclo

♦ Exercício 1.1 (Estilo Prova Real 2026 — 3 Operandos)

Instrução: `sub3 $rd, $rs, $rt`

Operação: $\$rd = \$rs - \$rt - \rd

A instrução realiza a subtração sucessiva de 3 registradores e grava o resultado final de volta em `$rd`.

1. Quantas portas de leitura o Banco de Registradores precisará ter? Explique quais campos da instrução (`Instruction[31:0]`) conectam em quais portas.
2. Desenhe/descreva as modificações necessárias no caminho de dados (ULAs adicionais, MUXes e fios).
3. Apresente a tabela com todos os sinais de controle (`RegDst`, `ALUSrc`, `MemtoReg`, `RegWrite`, `MemRead`, `MemWrite`, `Branch`, `ALUOp`, etc.).
4. **Cuidado de Prova:** Como evitar conflito de barramento (curto-circuito) na entrada `Write data` do banco de registradores?

♦ Exercício 1.2 (Estilo Prova Real 2026 — Processamento de Sinal / Módulo)

Instrução: `addabs $rd, $rs, $rt`

Operação: $\$rd = |\$rs + \$rt|$

A instrução calcula a soma de dois registradores e armazena o valor absoluto (módulo) do resultado em `$rd`.

1. Explique como o bloco combinacional de valor absoluto (`ABS`) deve ser inserido no datapath.
2. Como o multiplexador `MemtoReg` deve ser modificado (número de entradas e bits de controle) para suportar essa instrução sem quebrar instruções como `LW` e tipo R normais?
3. Defina a tabela de sinais de controle para a instrução `addabs`.

♦ Exercício 1.3 (Clássico dos Slides — Slide 4.2 / Patterson)

Instrução: `lwi $rt, $rs($rd)` (*Load Word Indexado*)

Operação: $\$rt = \text{Mem}[\$rs + \$rd]$

Em vez de somar um imediato de 16 bits à base, esta instrução soma o conteúdo de dois registradores (`$rs` e `$rd`) para formar o endereço de memória e grava o dado lido em `$rt`.

1. Indique quais registradores são lidos e qual é o registrador de destino de escrita.
2. Quais modificações são necessárias na entrada da ULA?
3. Monte a tabela de sinais de controle destacando os valores de `RegDst`, `ALUSrc` e `MemtoReg`.

♦ Exercício 1.4 (Instrução com Imediato Modificado)

Instrução: `addi_inc $rt, $rs, imm`

Operação: $\$rt = \$rs + \text{imm} + 1$

Soma o registrador `$rs` com o imediato estendido em sinal e adiciona mais uma unidade (+1) ao resultado.

1. Como implementar a adição da constante `+1` de forma eficiente no datapath? (Dica: utilize o `Carry-In` da ULA ou um somador).
2. Apresente a tabela de sinais de controle.

BLOCO 2: Projeto de Novas Instruções e FSM no Multiciclo

♦ Exercício 2.1 (Estilo Prova Real 2026 — Função de Ativação / Redes Neurais)

Instrução: `leaky_relu $rs`

Operação:

$$Se \quad r < 0 \quad \Rightarrow \quad r = 0$$

$$Se \quad r \geq 0 \quad \Rightarrow \quad r = r$$

Senão ($r < 0$) $\Rightarrow r = 0$

1. Qual modificação no multiplexador `RegDst` é obrigatória para permitir que o registrador `$rs` seja o destino da escrita?
2. Detalhe quais registradores internos do Multiciclo (`IR`, `A`, `B`, `ALUOut`, `MDR`) são utilizados em cada ciclo.
3. Desenhe/descreva os novos estados da Máquina de Estados Finitos (FSM) a partir do Estado 1 (Decodificação), listando todos os sinais de controle ativos em cada estado.
4. Quantos ciclos de clock essa instrução leva para ser executada em cada um dos ramos (positivo vs não-positivo)?

◆ Exercício 2.2 (Estilo Prova Real 2024 – Salto com Link / Procedimentos)

Instrução: `jal addr` (*Jump and Link no Multiciclo*)

Operação: `$31 = PC; \quad PC = PC[31:28] \parallel (IR[25:0] \ll 2)`

(Salva o endereço de retorno no registrador `$ra` / `$31` e realiza o salto para o endereço alvo).

1. Quais novos caminhos de dados e multiplexadores devem ser adicionados para conectar a constante `31` no endereço de escrita do Banco de Registradores e o valor do `PC` no barramento `Write data`?
2. A partir do **Estado 1** (Decodificação), crie o estado de conclusão do `JAL`.
3. Liste todos os sinais de controle ativos no ciclo de conclusão (`RegDst`, `MemtoReg`, `RegWrite`, `PCSource`, `PCWrite`).
4. Em quantos ciclos a instrução `JAL` é concluída?

◆ Exercício 2.3 (Instrução de Comparação Condicional)

Instrução: bgtz \$rs, offset (*Branch on Greater Than Zero*)

Operação: Se $\$rs > 0 \implies PC = PC + (\text{SignExt}(\text{imm}) \lll 2)$

1. No Ciclo 2 (Decodificação), o que a ULA calcula preventivamente?
2. No Ciclo 3 (Execução do Branch), como verificar se $\$rs > 0$ utilizando os sinais da ULA (bit de sinal e saída zero)?
3. Quais sinais de controle devem ser configurados no Ciclo 3 para atualizar o PC somente se a condição for verdadeira?

♦ **Exercício 2.4 (Instrução com Acesso Duplo à Memória)**

Instrução: swap (\$rs), (\$rt)

Operação: Troca os valores contidos nos dois endereços de memória apontados por \$rs e \$rt.

1. Quantos acessos à memória (leitura e escrita) são necessários?
2. Quantos ciclos de clock no total o processador multiciclo gastará para executar essa instrução?
3. Liste o RTL (transferência entre registradores) passo a passo para cada ciclo.

 BLOCO 3: Análise de Sinais e Caminho Crítico

♦ **Exercício 3.1 (Tabela Completa de Sinais do Multiciclo)**

Preencha a tabela de sinais de controle do processador Multiciclo para cada um dos passos das seguintes instruções:

Instrução	Ciclo	MemRead	MemWrite	IorD	IRWrite	RegWrite	RegDst	MemtoReg	ALUSrcA	ALUSrcB	ALUOp	F
Busca (Fetch)	1											
Decodificação	2											
ADD (Execução)	3											
ADD (Write-back)	4											
LW (Calc Endereço)	3											
LW (Leitura Mem)	4											
LW (Write-back)	5											
BEQ (Conclusão)	3											

♦ **Exercício 3.2 (Cálculo de Desempenho, CPI e Speedup)**

Considere as seguintes latências dos componentes de hardware:

- **Memória (I-Mem / D-Mem):** 250 ps
- **ULA:** 200 ps
- **Leitura/Escrita no Banco de Registradores:** 150 ps
- **Extensor de Sinal / MUXes / Fios:** Desprezíveis (0 ps)

Considere um programa com a seguinte distribuição de instruções:

- **Tipo R (ADD, SUB, etc.):** 45%
- **Load (LW):** 25%
- **Store (SW):** 15%

- **Branch (BEQ):** 10%
- **Jump (J):** 5%

Pede-se:

1. Determine o **tempo de ciclo de clock** (T_{clk}) do processador **Monociclo** (baseado na instrução mais lenta).
2. Determine o **tempo de ciclo de clock** (T_{clk}) do processador **Multiciclo** (baseado no estágio mais lento).
3. Calcule o **CPI Médio** do processador Multiciclo:

$$CPI_{\text{médio}} = \sum (Mix_i \times Ciclos_i)$$

4. Calcule o **Speedup** do Multiciclo sobre o Monociclo:

$$Speedup = \frac{\text{Tempo Monociclo}}{\text{Tempo Multiciclo}} = \frac{CPI_{\text{mono}} \times T_{clk, \text{mono}}}{CPI_{\text{multi}} \times T_{clk, \text{multi}}}$$

5. O processador Multiciclo foi mais rápido ou mais lento que o Monociclo para este programa? Justifique o resultado.